Python OOP, learned *live*

Every module, demo, quiz and challenge from the Interactive Learning Studio, run in a real browser and captured as it happened.

3/4 Object Builder
Classes and Objects

4/4 Guardian
Encapsulation

4/4 Heir
Inheritance

3/4 Shapeshifter
Polymorphism

4/5 Architect
Final challenge

500 XP
earned in total

100%
course complete

18 / 21
quiz and challenge answers correct

0
console errors in the run

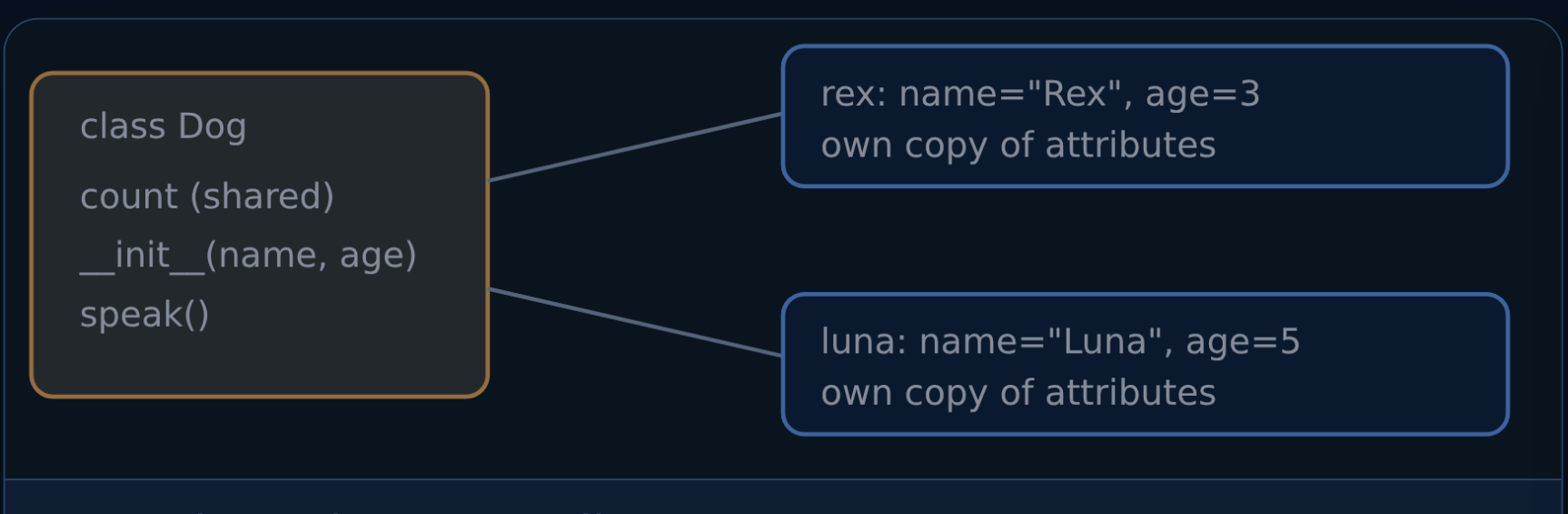
Module 1 Classes and Objects

Failed first: 2/4

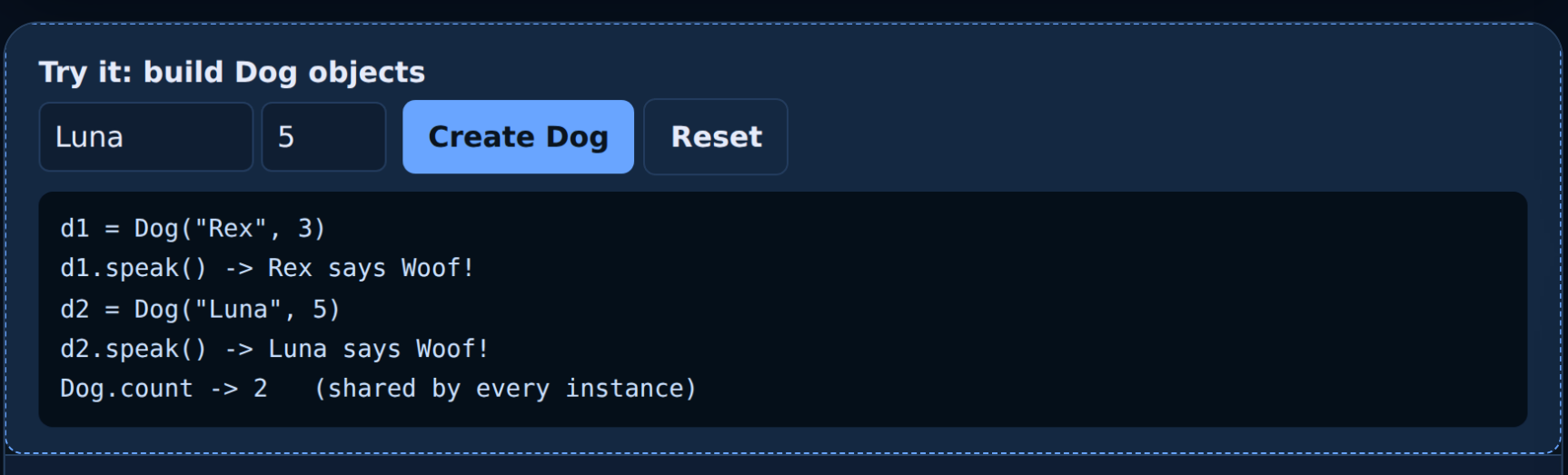
Retry: 3/4, +75 XP

The gate worked: with 2 of 4 the Next button and Module 2 stayed locked. A retry at 3 of 4 unlocked them.

```
d1 = Dog("Rex", 3)
d2 = Dog("Luna", 5)
Dog.count -> 2   (shared)
```

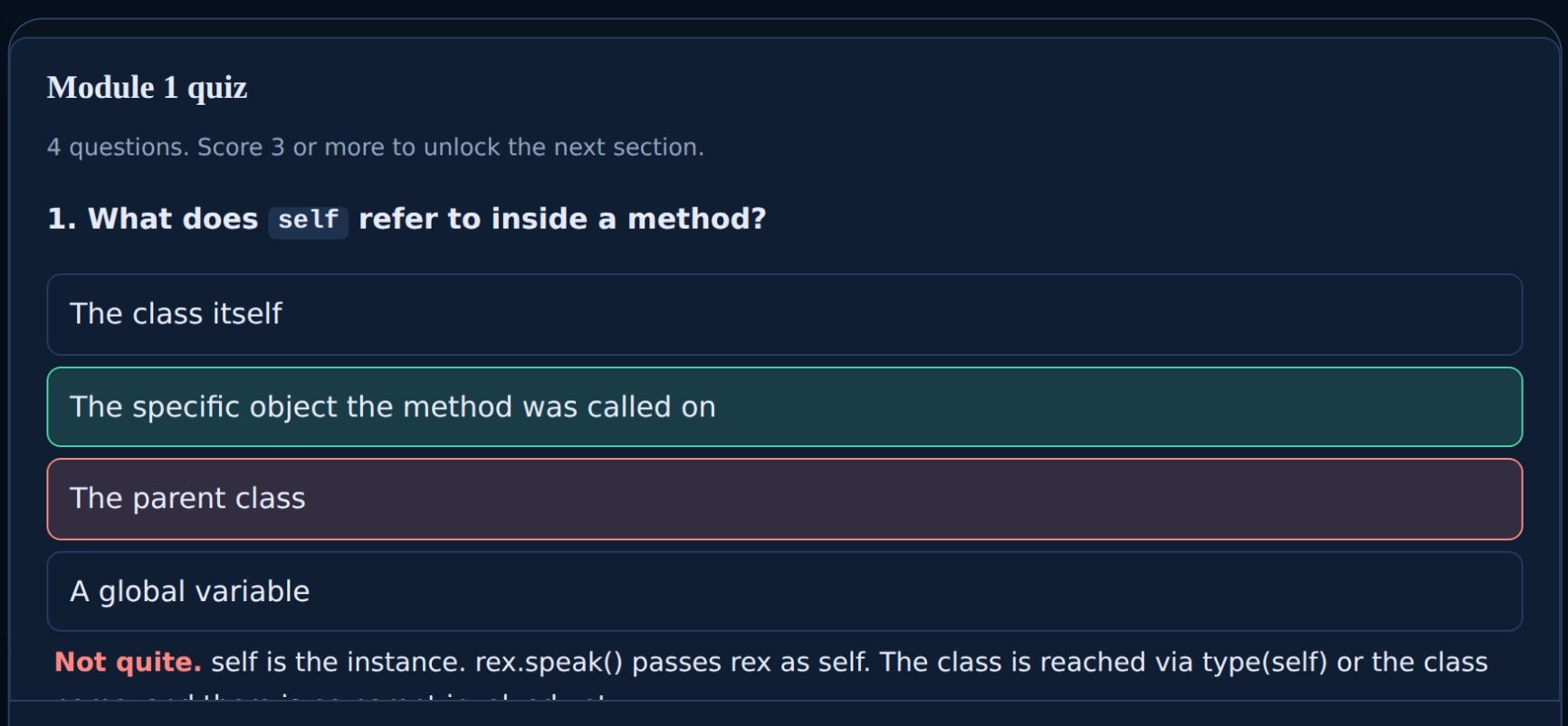


Blueprint to instances diagram



```
d1 = Dog("Rex", 3)
d1.speak() -> Rex says Woof!
d2 = Dog("Luna", 5)
d2.speak() -> Luna says Woof!
Dog.count -> 2   (shared by every instance)
```

Demo: two Dog objects created



Module 1 quiz

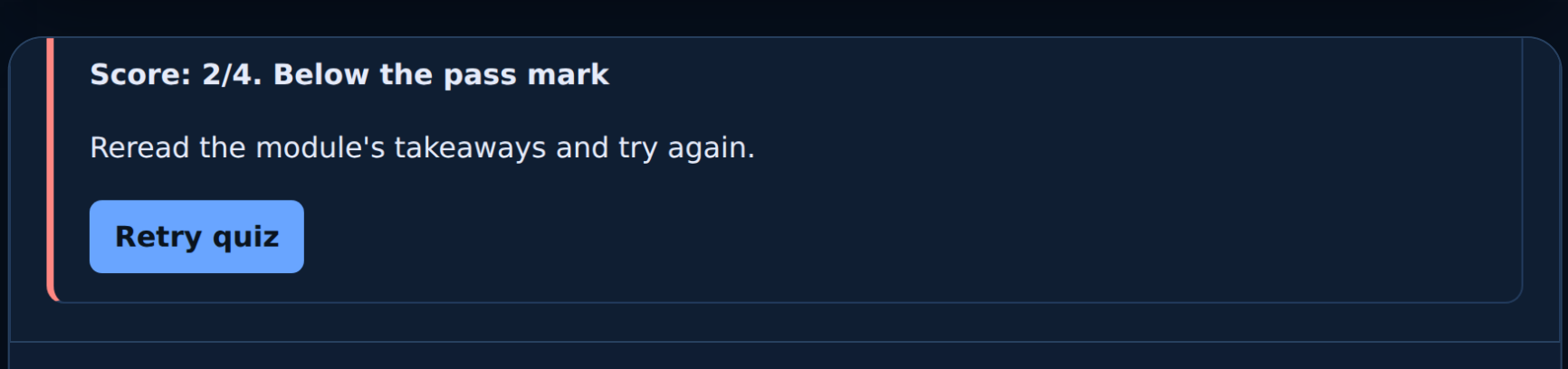
4 questions. Score 3 or more to unlock the next section.

1. What does `self` refer to inside a method?

- The class itself
- The specific object the method was called on
- The parent class
- A global variable

Not quite. self is the instance. rex.speak() passes rex as self. The class is reached via type(self) or the class

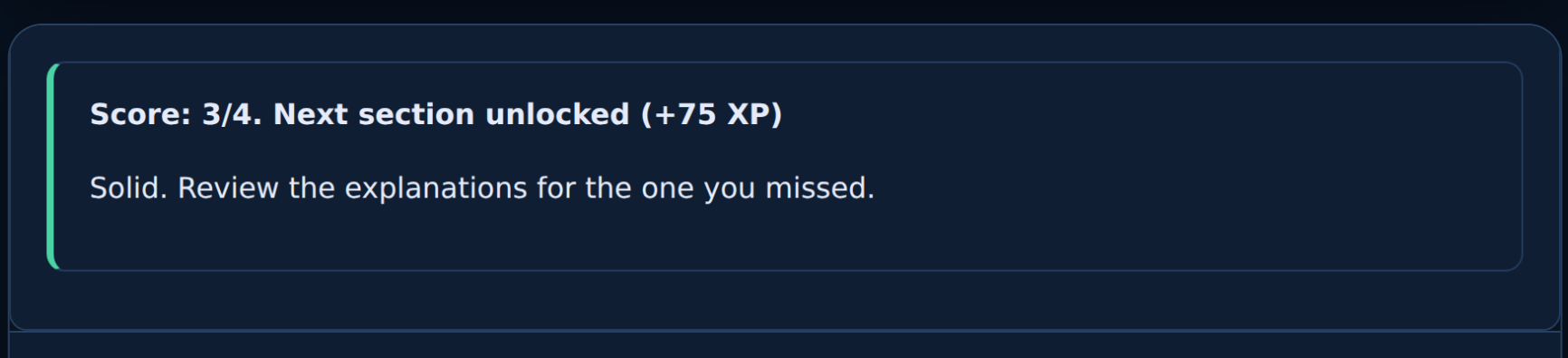
Quiz question with instant feedback



Score: 2/4. Below the pass mark

Reread the module's takeaways and try again.

First attempt: below pass mark, locked



Score: 3/4. Next section unlocked (+75 XP)

Solid. Review the explanations for the one you missed.

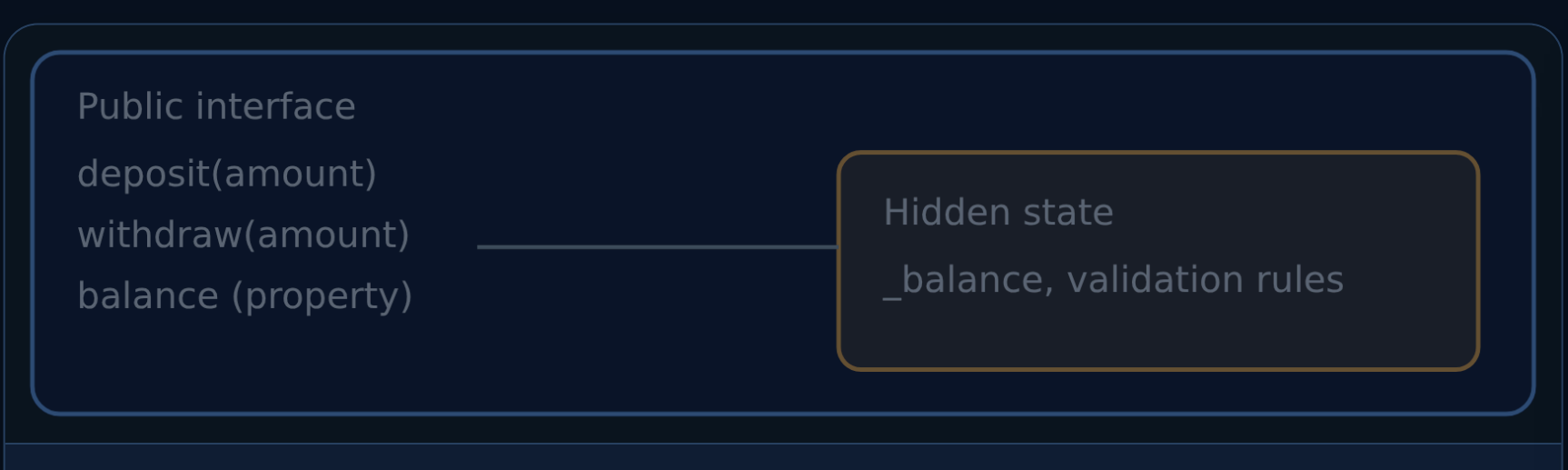
Retry: unlocked, +75 XP

Module 2 Encapsulation and Properties

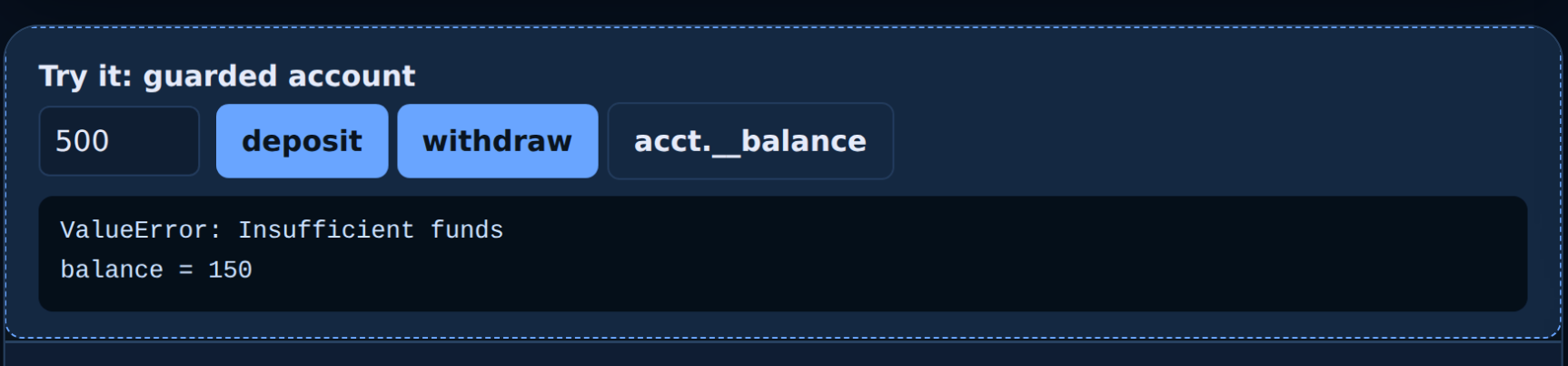
Quiz 4/4, +100 XP

The guarded account accepted a valid deposit and rejected an overdraw with a clear error, just as the lesson teaches.

```
deposit(50)    ok
withdraw(500) ValueError:
              Insufficient funds
balance = 150
```

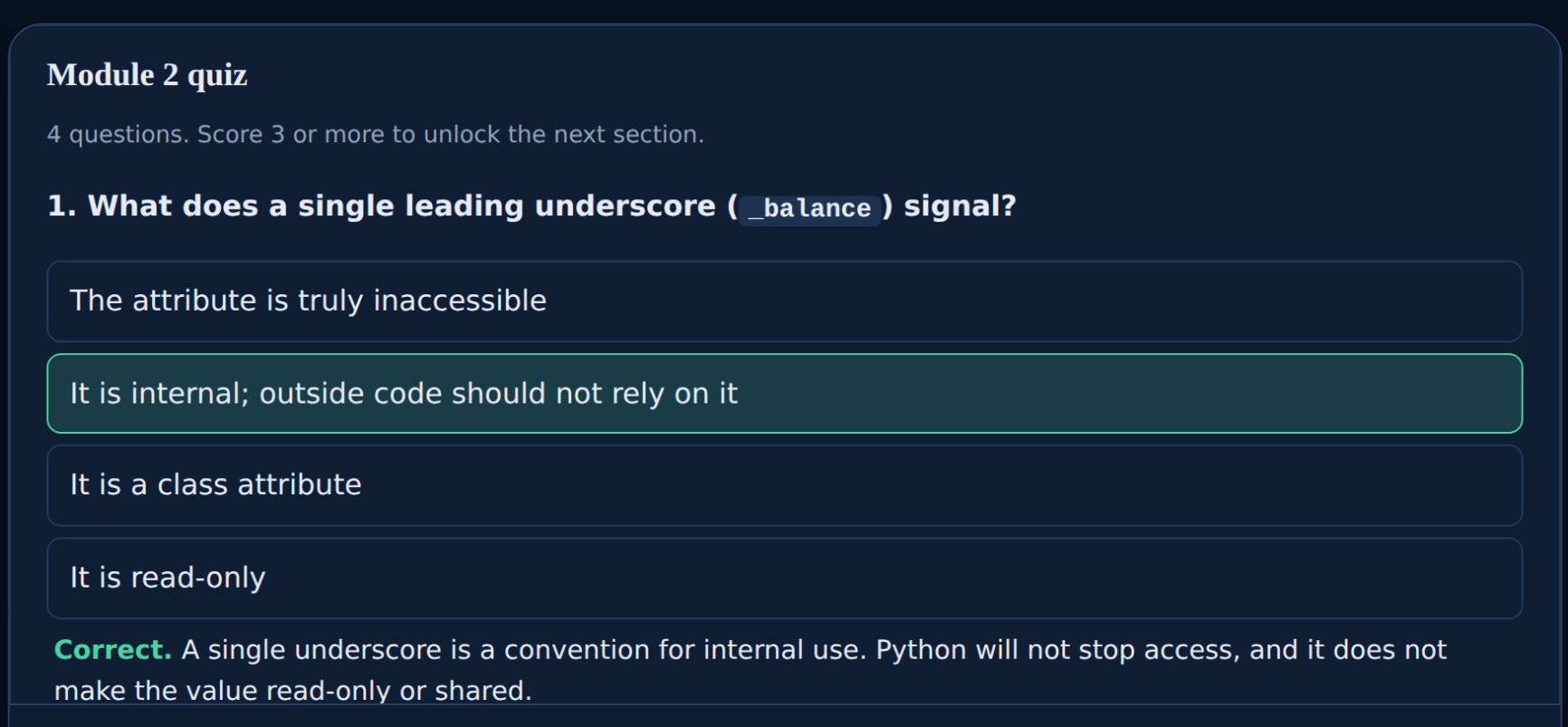


Public interface around hidden state



```
ValueError: Insufficient funds
balance = 150
```

Demo: guarded account



Module 2 quiz

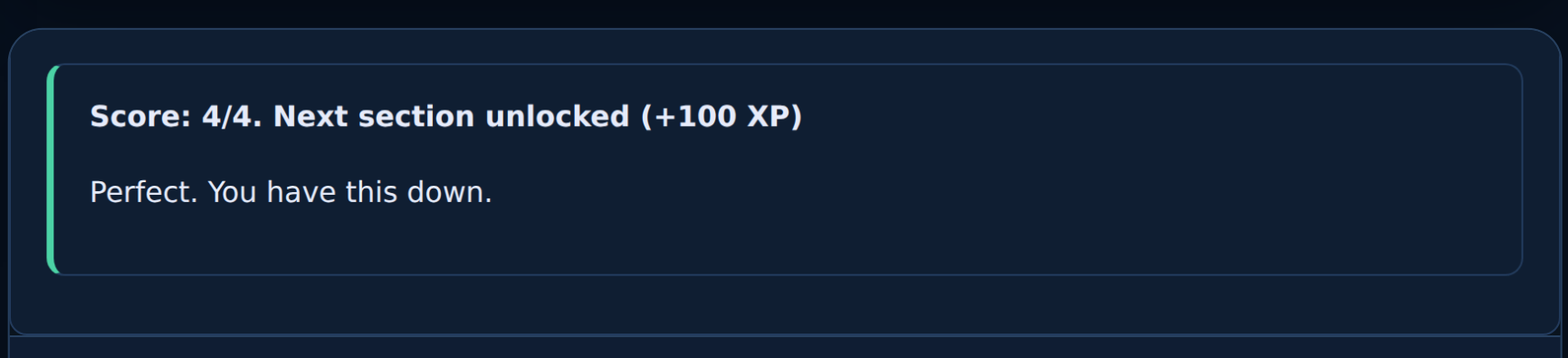
4 questions. Score 3 or more to unlock the next section.

1. What does a single leading underscore (`_balance`) signal?

- The attribute is truly inaccessible
- It is internal; outside code should not rely on it
- It is a class attribute
- It is read-only

Correct. A single underscore is a convention for internal use. Python will not stop access, and it does not make the value read-only or shared.

Quiz question with explanation



Score: 4/4. Next section unlocked (+100 XP)

Perfect. You have this down.

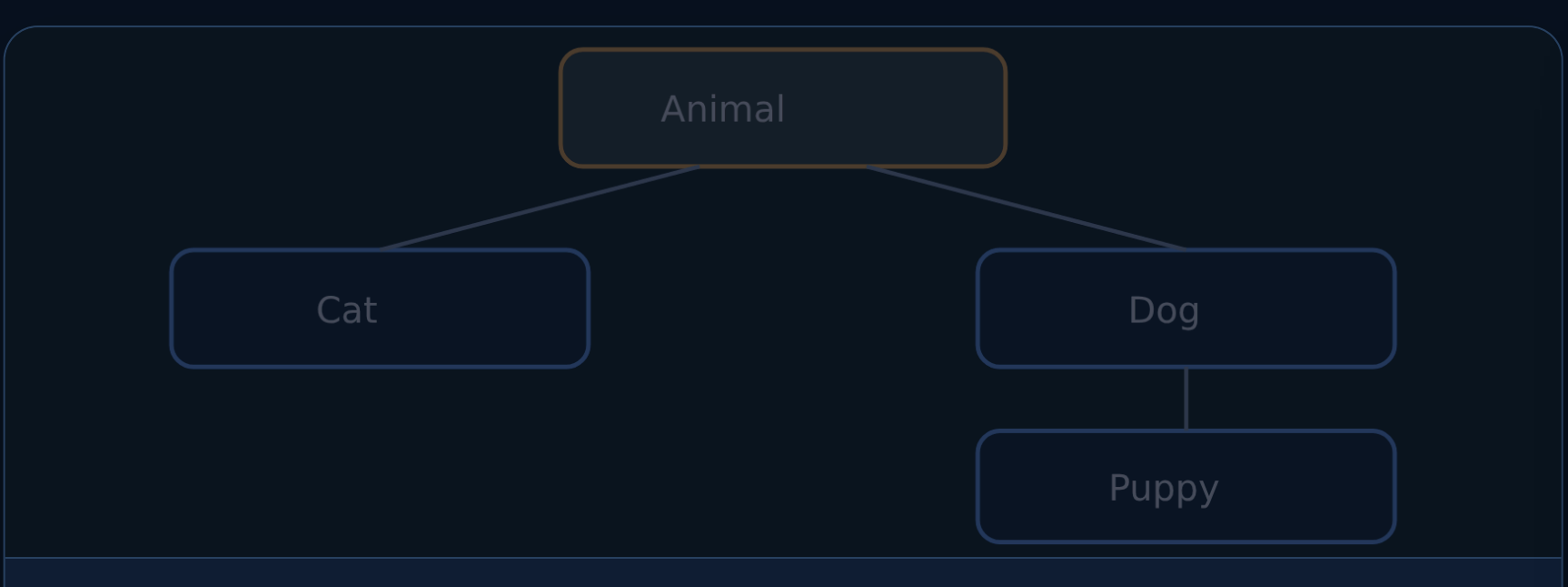
Perfect score, next section unlocked

Module 3 Inheritance and Composition

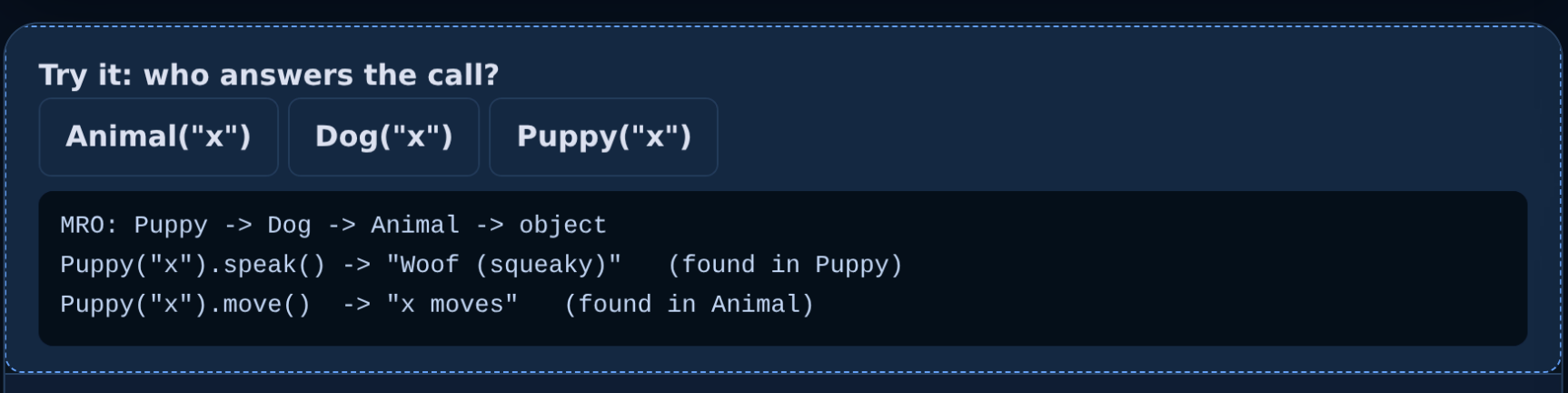
Quiz 4/4, +100 XP

Picking Puppy showed the method resolution order and which class answers each call.

```
MRO: Puppy -> Dog -> Animal
       -> object
speak() found in Puppy
move()  found in Animal
```

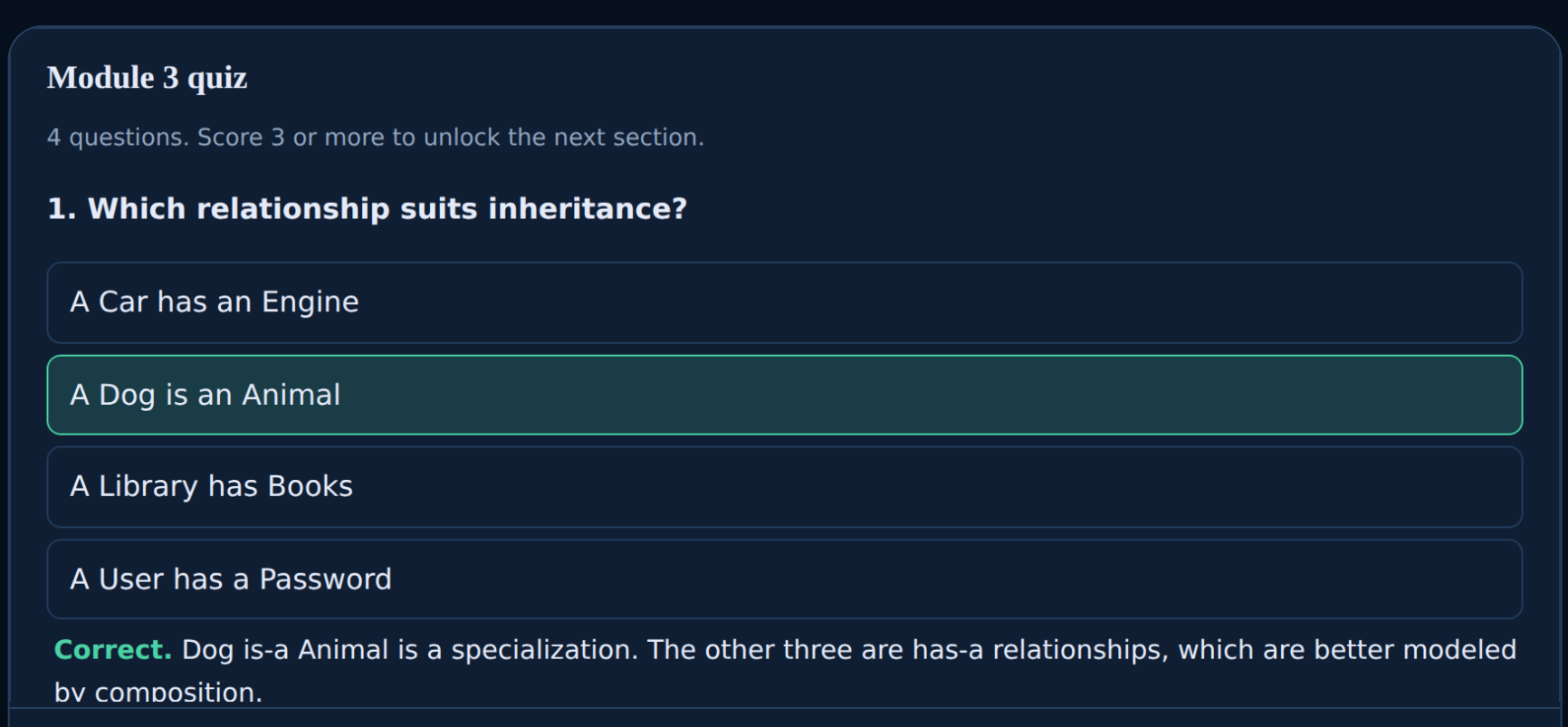


Animal hierarchy diagram



```
MRO: Puppy -> Dog -> Animal -> object
Puppy("x").speak() -> "Woof (squeaky)"   (found in Puppy)
Puppy("x").move()  -> "x moves"   (found in Animal)
```

Demo: who answers the call?



Module 3 quiz

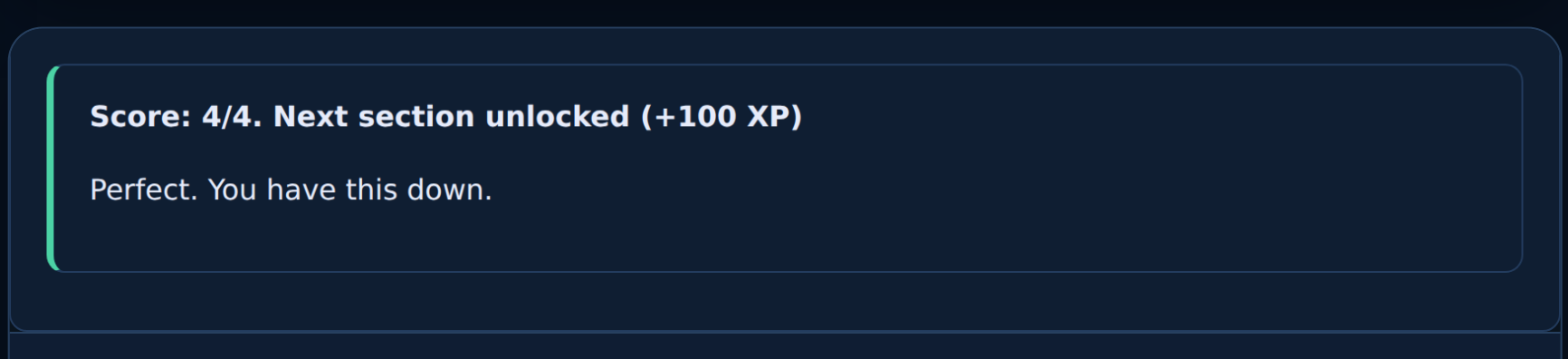
4 questions. Score 3 or more to unlock the next section.

1. Which relationship suits inheritance?

- A Car has an Engine
- A Dog is an Animal
- A Library has Books
- A User has a Password

Correct. Dog is-a Animal is a specialization. The other three are has-a relationships, which are better modeled by composition.

Quiz question with explanation



Score: 4/4. Next section unlocked (+100 XP)

Perfect. You have this down.

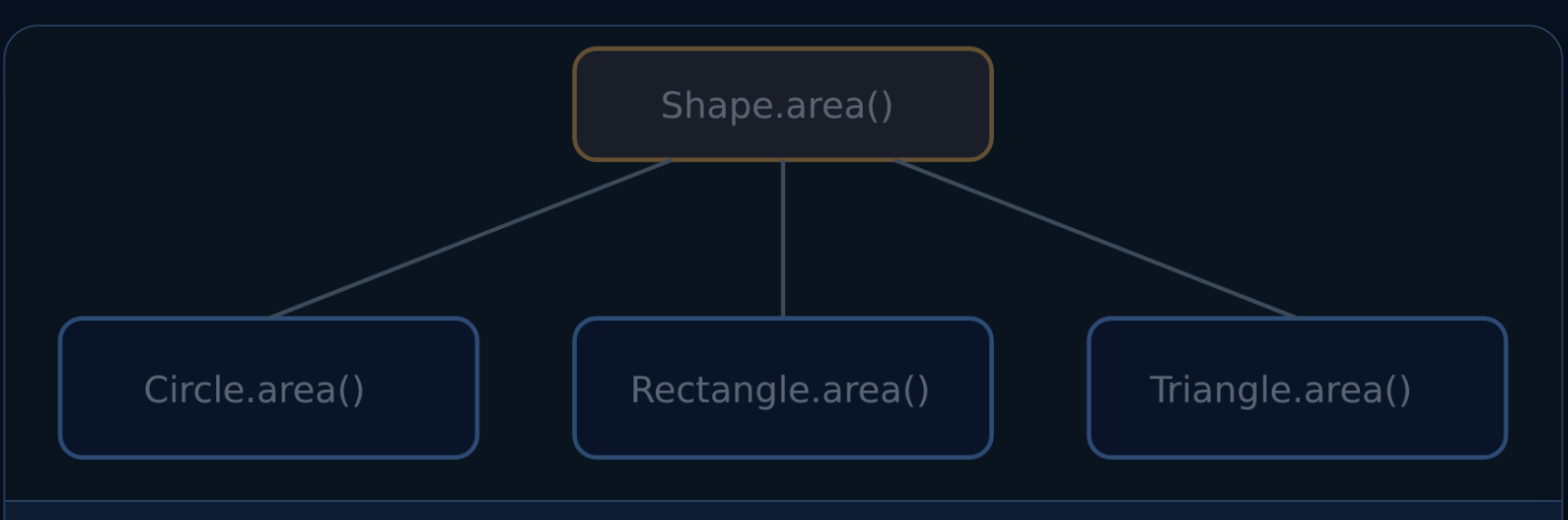
Perfect score, next section unlocked

Module 4 Polymorphism and Abstraction

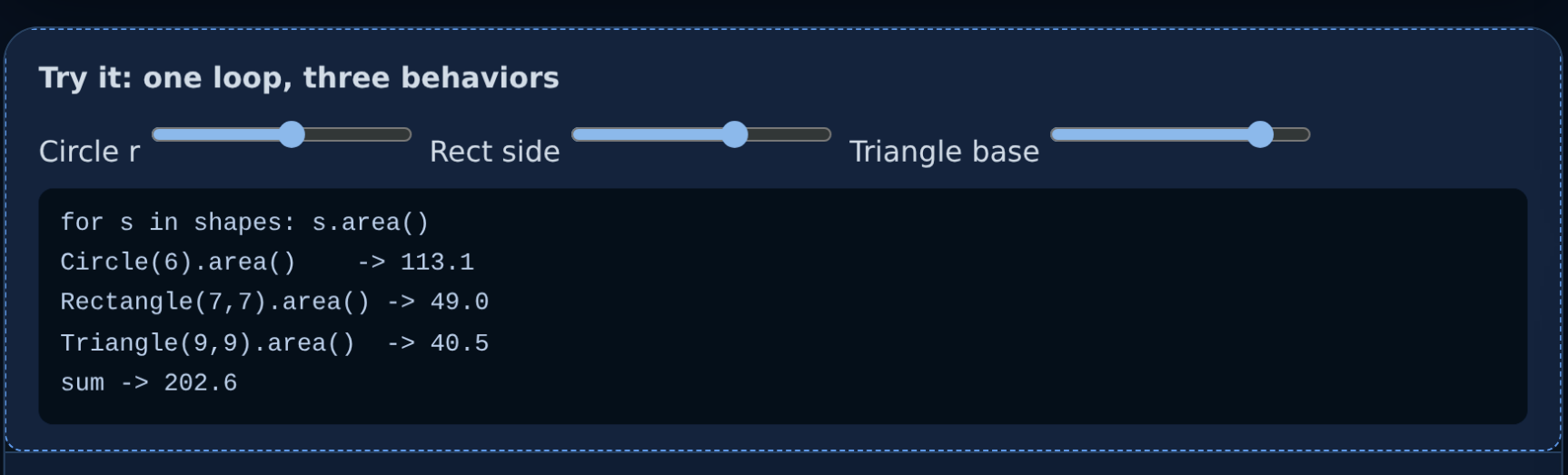
Quiz 3/4, +75 XP

One loop called `area()` on three different shapes. Sliders were set to 6, 7 and 9.

```
Circle(6).area()      113.1
Rectangle(7,7).area()  49.0
Triangle(9,9).area()   40.5
sum                   202.6
```

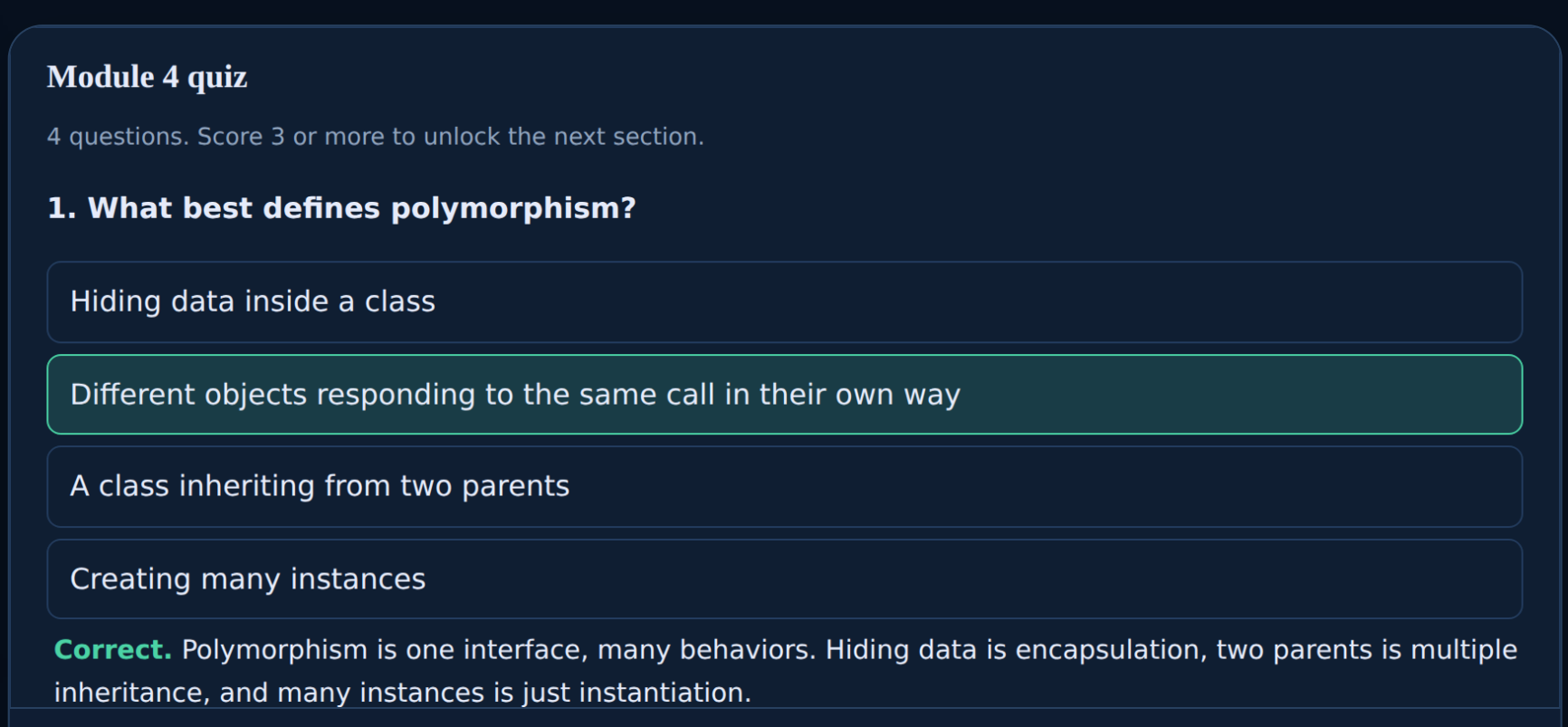


Shape interface with three implementations



```
for s in shapes: s.area()
Circle(6).area()    -> 113.1
Rectangle(7,7).area() -> 49.0
Triangle(9,9).area()  -> 40.5
sum -> 202.6
```

Demo: one loop, three behaviors



Module 4 quiz

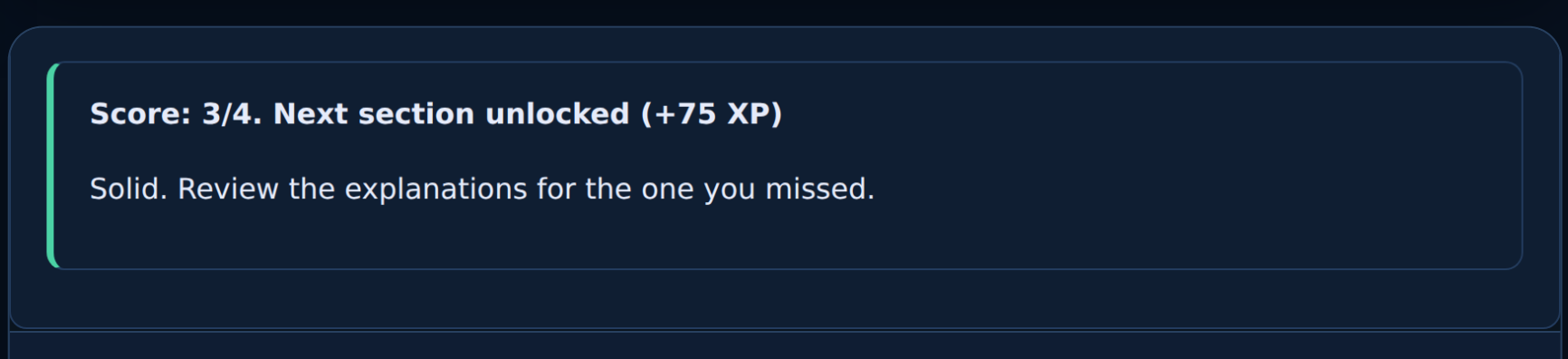
4 questions. Score 3 or more to unlock the next section.

1. What best defines polymorphism?

- Hiding data inside a class
- Different objects responding to the same call in their own way
- A class inheriting from two parents
- Creating many instances

Correct. Polymorphism is one interface, many behaviors. Hiding data is encapsulation, two parents is multiple inheritance, and many instances is just instantiation.

Quiz question with explanation



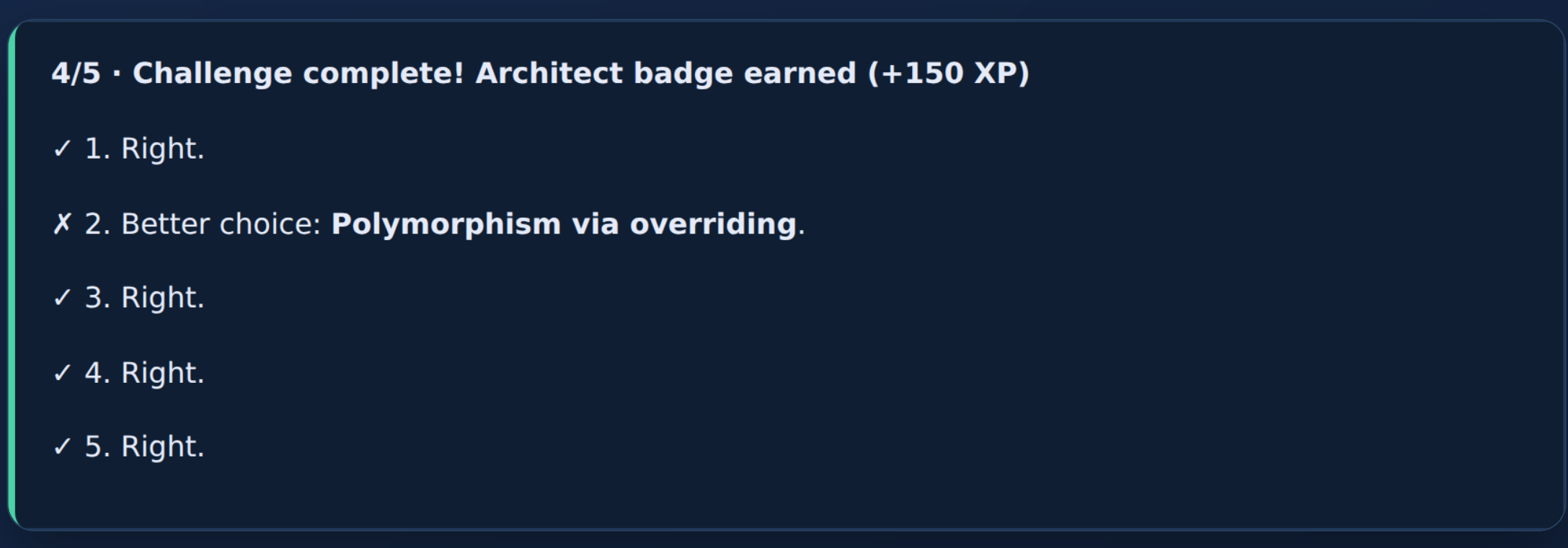
Score: 3/4. Next section unlocked (+75 XP)

Solid. Review the explanations for the one you missed.

Passed, course fully unlocked

Final challenge: design a library system

Five design choices, scored 4 of 5 with one deliberate miss. The app explained the better answer (polymorphism through overriding) and still awarded the Architect badge and +150 XP.



4/5 · Challenge complete! Architect badge earned (+150 XP)

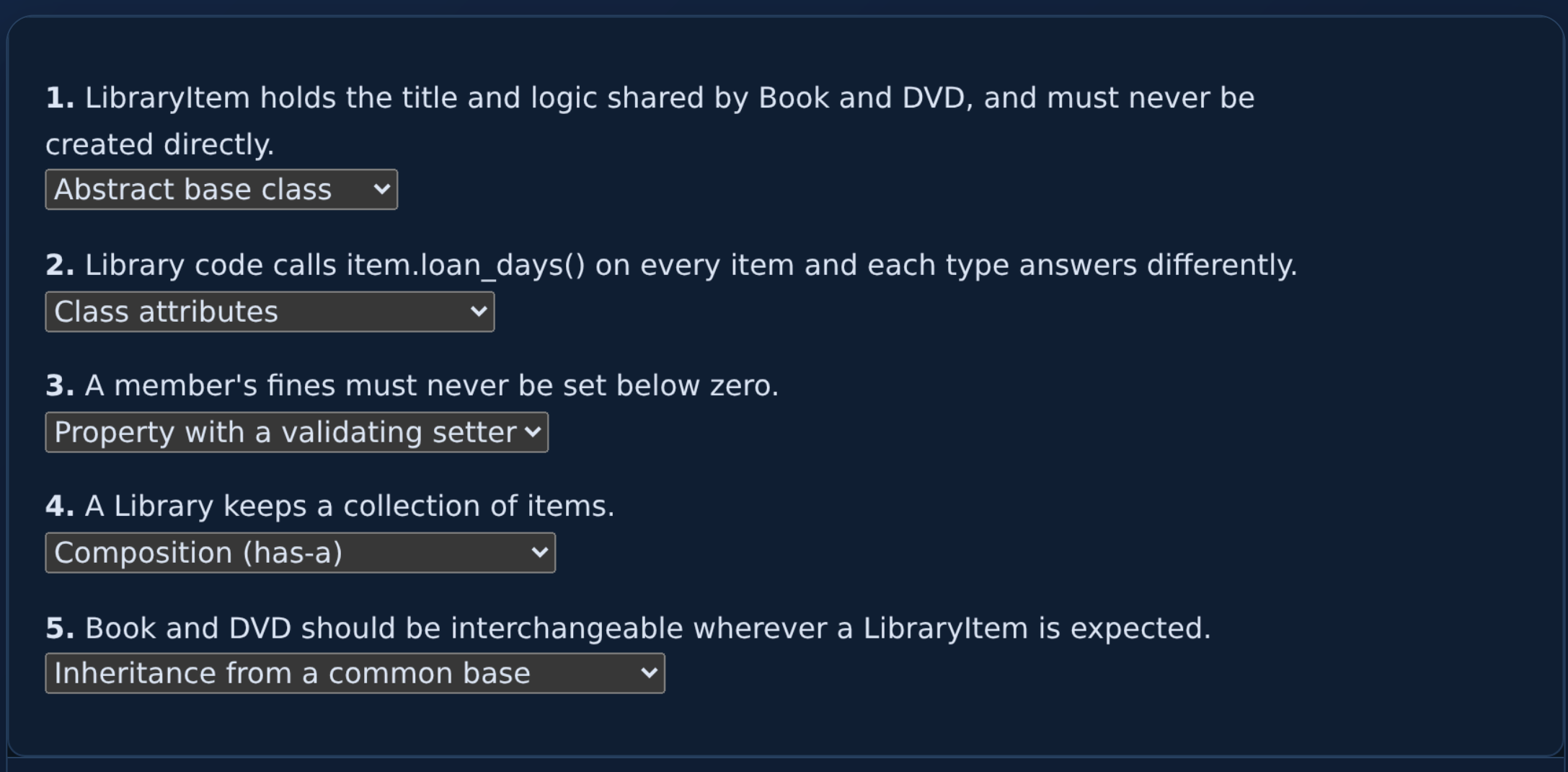
✓ 1. Right.

✗ 2. Better choice: **Polymorphism via overriding**.

✓ 3. Right.

✓ 4. Right.

✓ 5. Right.



1. LibraryItem holds the title and logic shared by Book and DVD, and must never be created directly.
Abstract base class

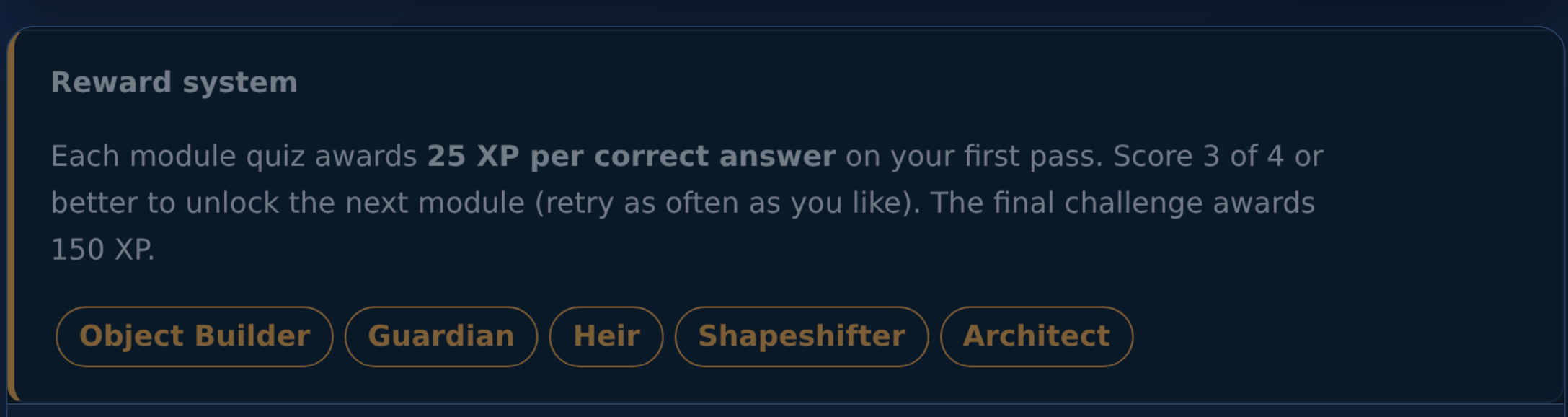
2. Library code calls item.loan_days() on every item and each type answers differently.
Class attributes

3. A member's fines must never be set below zero.
Property with a validating setter

4. A Library keeps a collection of items.
Composition (has-a)

5. Book and DVD should be interchangeable wherever a LibraryItem is expected.
Inheritance from a common base

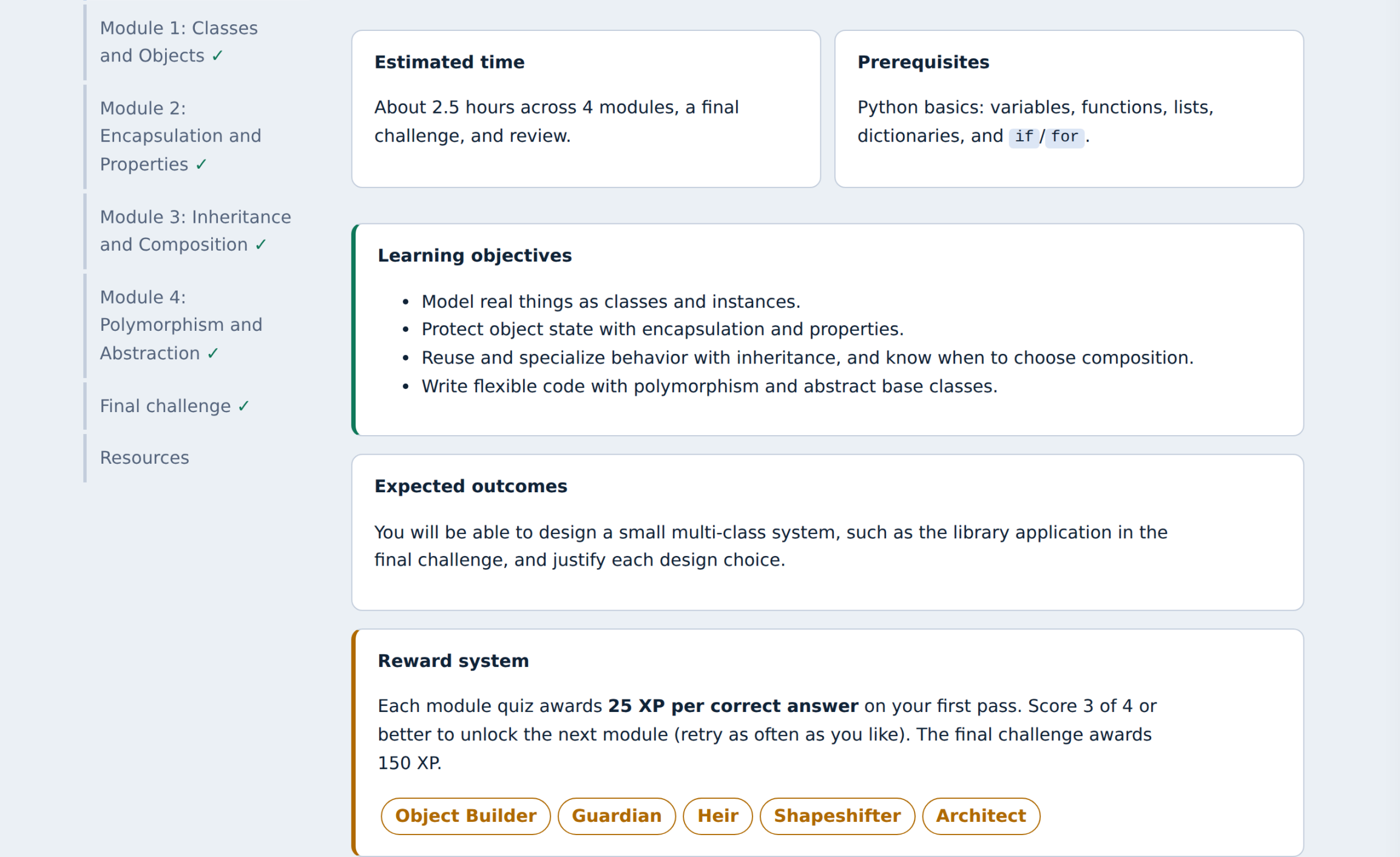
The five design questions



Reward system

Each module quiz awards **25 XP per correct answer** on your first pass. Score 3 of 4 or better to unlock the next module (retry as often as you like). The final challenge awards 150 XP.

Reward system on the introduction page


Interactive Learning Studio · Python OOP
100% complete
500 XP
Dark mode
Print notes
Introduction
Module 1: Classes and Objects ✓
Module 2: Encapsulation and Properties ✓
Module 3: Inheritance and Composition ✓
Module 4: Polymorphism and Abstraction ✓
Final challenge ✓
Resources

Classes, inheritance, and polymorphism, taught from first principles with live demos, exercises, and quizzes.

Estimated time

About 2.5 hours across 4 modules, a final challenge, and review.

Prerequisites

Python basics: variables, functions, lists, dictionaries, and `if` / `for`.

Learning objectives

- Model real things as classes and instances.
- Protect object state with encapsulation and properties.
- Reuse and specialize behavior with inheritance, and know when to choose composition.
- Write flexible code with polymorphism and abstract base classes.

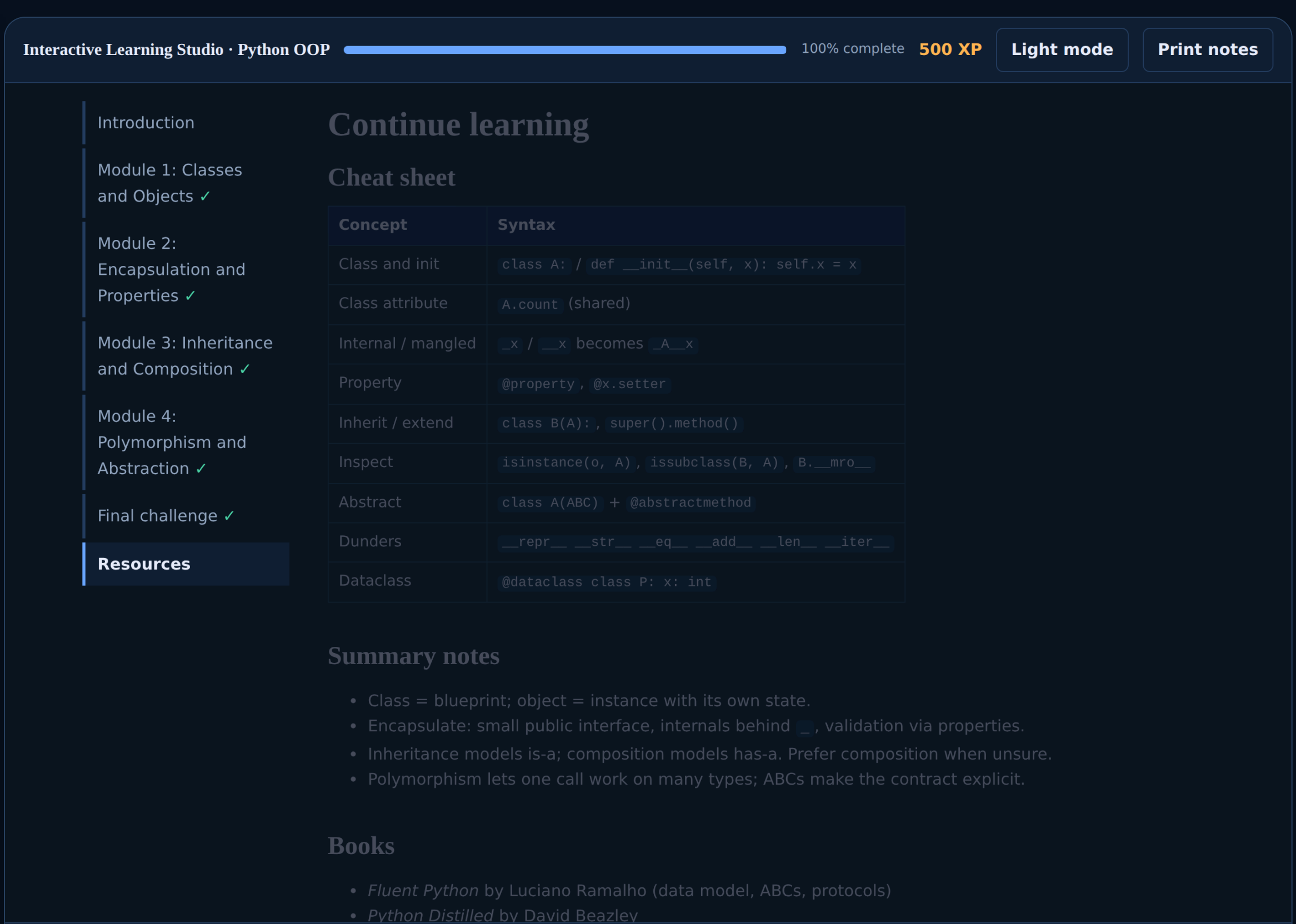
Expected outcomes

You will be able to design a small multi-class system, such as the library application in the final challenge, and justify each design choice.

Reward system

Each module quiz awards **25 XP per correct answer** on your first pass. Score 3 of 4 or better to unlock the next module (retry as often as you like). The final challenge awards 150 XP.

Light mode


Interactive Learning Studio · Python OOP
100% complete
500 XP
Light mode
Print notes
Introduction
Module 1: Classes and Objects ✓
Module 2: Encapsulation and Properties ✓
Module 3: Inheritance and Composition ✓
Module 4: Polymorphism and Abstraction ✓
Final challenge ✓
Resources

Continue learning

Cheat sheet

Concept	Syntax
Class and init	`class A:` / `def __init__(self, x): self.x = x`
Class attribute	`A.count` (shared)
Internal / mangled	`_x` / `__x` becomes `_A__x`
Property	`@property` , `@x.setter`
Inherit / extend	`class B(A):` , `super().method()`
Inspect	`isinstance(o, A)` , `issubclass(B, A)` , `B.__mro__`
Abstract	`class A(ABC)` + `@abstractmethod`
Dunders	`__repr__ __str__ __eq__ __add__ __len__ __iter__`
Dataclass	`@dataclass class P: x: int`

Summary notes

- Class = blueprint; object = instance with its own state.
- Encapsulate: small public interface, internals behind `_`, validation via properties.
- Inheritance models is-a; composition models has-a. Prefer composition when unsure.
- Polymorphism lets one call work on many types; ABCs make the contract explicit.

Books

- *Fluent Python* by Luciano Ramalho (data model, ABCs, protocols)
- *Python Distilled* by David Beazley

Resources, cheat sheet and AI prompts

Gating verified

Modules 2 to 4, the final challenge and Resources stayed locked until the previous quiz was passed.

Scoring verified

Fail, retry and pass all produced the correct score, message and XP. Total: 75 + 100 + 100 + 75 + 150 = 500.

Display verified

Dark and light themes render, and the 390 px mobile layout has no sideways scroll.

Interactive Learning Studio

Python Object-Oriented Programming · classes, inheritance, polymorphism